PROJECT WORK | EMBEDDED SYSTEMS + INTERNET OF THINGS

PlantPal

IoT-Enabled Smart Plant Monitoring and Voice Assistance System

A working ESP32 prototype centred on one Tulsi plant



PROGRAMME

S.Y. B.Sc. IT
K. E. S. Shroff College (Autonomous)

ACADEMIC YEAR

2026 - 27
Project Work Submission

PROJECT TEAM

Aashutosh Mishra	SDIT060A
Akshat Chheda	SDIT011A
Vivek Dubey	SDIT016A

PROJECT GUIDES

Mr. Aryan Mourya
Mr. Sumit Singh

SUBMISSION DATE

01 October 2026

Table of Contents

SECTION	PAGE
Chapter 1 - Introduction	3 - 4
Chapter 2 - Literature Review	5 - 6
Chapter 3 - Proposed System	7 - 8
Chapter 4 - Requirements	9 - 10
Chapter 5 - Methodology & Implementation	11 - 12
Chapter 6 - Experimental Setup & Results	13
Chapter 7 - Applications, Advantages & Limitations	14
Chapter 8 - Future Scope	14
Chapter 9 - Conclusion	14
References & Appendix	15

AT A GLANCE

CONTROLLER
ESP32 Dev Module

PLANT
Tulsi

INPUTS
Soil + DHT11 + BH1750 + Touch

OUTPUTS
OLED + DFPlayer + Speaker + Buzzer

IOT
Wi-Fi + HTTPS/REST + Flask + SQLite

VOICE
73 stored voice tracks

WATERING
Human-controlled; no pump

"Technology that listens to the plant."

A compact project theme: measure, interpret, communicate, and keep the human in control.

1 Introduction

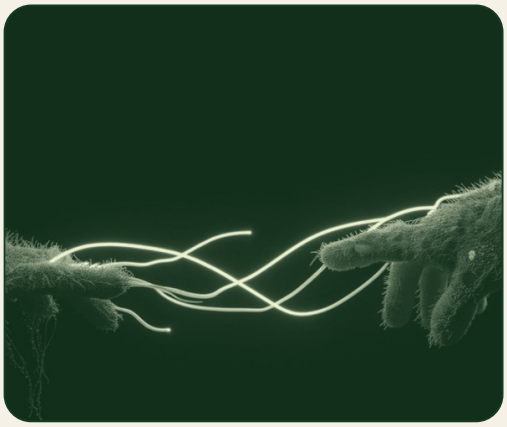
1.1 Introduction

PlantPal is a compact embedded system built around an ESP32 and a living Tulsi plant. Instead of treating each sensor as an isolated reading, the prototype combines soil moisture, ambient light, temperature and humidity into a single plant-care interface. The local device presents information on a 128 x 64 OLED, accepts touch input, and communicates spoken feedback through a DFPlayer Mini and speaker.

The IoT layer extends the same device to a browser dashboard. Sensor telemetry is sent from the ESP32 over Wi-Fi using HTTPS/REST to a Flask backend with SQLite storage. Remote commands are queued in the cloud and polled by the device. The embedded loop is intentionally local-first, so plant monitoring and interaction are not dependent on Internet availability.

Why PlantPal matters as a college prototype

The project connects physical sensing, local interaction, voice feedback and a two-way IoT path in one enclosure. The purpose is not autonomous agriculture; it is a clear demonstration of embedded systems working together with an IoT service.



HUMAN-IN-THE-LOOP

The caretaker remains responsible for watering. PlantPal measures the environment, communicates what the measurements mean, and lets the user act.

Figure 1.1: Conceptual visual for the project principle - technology supporting everyday plant care. Reference image supplied by the project author; colour treatment adapted for this report.

SENSE
Soil + light + climate

INTERPRET
States + health score

COMMUNICATE
OLED + voice + web

1 Introduction | Objectives, Scope & Problem

1.2 Problem Statement

Manual plant care often depends on occasional observation and personal judgement. A caretaker may know that a plant looks “fine” while missing a change in root-zone moisture, ambient light or microclimate. A small embedded monitor can make those changes visible without requiring constant inspection.

Design question

How can a low-cost embedded device sense the plant environment, interpret those readings, communicate the result locally, and make that information available remotely?

1.3 Aim

To develop and document a low-cost IoT-enabled smart plant system that monitors the condition of a Tulsi plant and provides understandable local and remote feedback through embedded interfaces and a web dashboard.

1.4 Objectives

01
Sense
Acquire soil moisture, light intensity, temperature and humidity.

02
Interpret
Convert readings into project-defined states and a composite health score.

03
Interact
Provide OLED pages, touch actions and spoken event feedback.

04
Connect
Upload telemetry to a cloud backend and receive queued commands.

05
Document
Demonstrate the complete hardware-software path as a B.Sc. IT project.

1.5 Scope

- One Tulsi plant in a two-layer pot enclosure;
- Embedded monitoring continues locally even without Internet;
- Cloud layer provides telemetry, event history and remote commands;
- Human-controlled watering; no automatic pump or irrigation actuator;
- Soil percentage remains a project-specific calibrated relative scale.

PLANT PROFILE

Tulsi
Ocimum tenuiflorum

Perennial / subshrub in tropical and subtropical Asia.
Taxonomic identity cross-checked against Kew Plants of the World Online.

2 Literature Review

2.1 Existing Systems

Plant-monitoring research increasingly combines environmental sensing with networked visualisation. The literature includes ESP-based monitors, cloud dashboards, automated irrigation, and larger agriculture-oriented systems. PlantPal is positioned within this established area, but uses the same physical device for local sensing, local feedback and a two-way cloud control path.

Table 2.1 | Selected related studies

Study	Core approach	What it contributes to this review
Absar et al. (2023) [8]	IoT plant monitoring using temperature, humidity and soil moisture, with mobile/cloud visibility and irrigation control.	Shows the value of multi-parameter sensing and remote access.
ICETET-SIP (2022) [9]	IoT smart-plant monitoring with cloud visualisation and moisture-based pump control.	Provides a direct comparison with cloud-connected irrigation systems.
Appl. Sci. (2023) [10]	IoT-SCADA monitoring of air and soil parameters with a cloud HMI.	Demonstrates broader sensor integration and networked environmental monitoring.
ICISC (2025) [11]	ESP32 indoor plant health monitoring with Wi-Fi and concurrent task handling.	Shows a recent ESP32-focused direction for indoor plant monitoring.

2.2 Review of Related Research Papers

Across these studies, common building blocks are soil/environment sensors, a microcontroller, wireless connectivity, and a user-facing monitoring layer. More advanced systems add automated irrigation, SCADA-style dashboards, or real-time task scheduling. PlantPal keeps a smaller embedded loop and human-controlled watering.

Research gap used for this project

The gap is best treated as an integration problem rather than a claim of novel sensing: combining multiple low-cost sensors, local OLED/touch/voice feedback and a bidirectional cloud command path in a compact academic planner

2 Literature Review | Comparative Study

2.3 Comparative Study

Feature	Traditional care	Typical IoT monitor	PlantPal
Continuous sensing	Low / periodic	Yes	Yes - timed local reads
Local display	No	Sometimes	OLED 128 x 64
Voice feedback	No	Rare	73-track stored voice library
Remote monitoring	No	Yes	Render web dashboard
Remote commands	No	Varies	Queued HTTPS commands
Automatic watering	Manual	Often included	No - human controlled
Plant-specific profile	Human knowledge	General / configurable	Tulsi profile
Offline local function	Not applicable	Varies	Local sensing + UI continue

2.4 Research Gap

Most related systems focus on measurement, cloud monitoring, or automation as the main story. PlantPal uses the same underlying IoT techniques, but its project value comes from integration: a compact embedded device interprets sensor readings locally, provides direct physical feedback, and synchronises that state with a web application.

THE DESIGN IN ONE LINE

SENSE
soil + light + climate + touch

INTERPRET
state labels + composite health

COMMUNICATE
OLED + voice + web dashboard

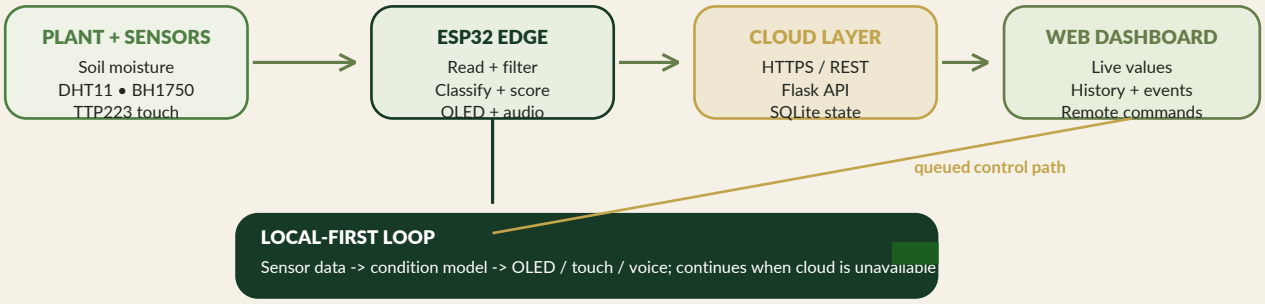
The selected literature supports the project context; the reported PlantPal behaviour is based on the supplied firmware, backend and test records.

3 Proposed System

3.1 Proposed Solution

PlantPal uses the ESP32 as the centre of a local sensing-and-response loop. Four sensing inputs feed one controller; the controller validates the values, classifies the current plant condition, updates the OLED, handles touch gestures, controls audio, and then synchronises selected state to the cloud when Wi-Fi is available.

Figure 3.1 | PlantPal system architecture



3.2 System Architecture

The architecture is deliberately layered. The plant and sensors form the physical layer; the ESP32 is the edge-processing layer; OLED, touch, DFPlayer and buzzer form the local interaction layer; and Wi-Fi, HTTPS/REST, Flask and SQLite form the IoT extension.

BIDIRECTIONAL IOT

- Telemetry: ESP32 -> cloud
- Commands: cloud -> ESP32
- Command confirmations + events: ESP32 -> cloud

DESIGN CHOICES

Local-first

Core sensing and UI work without the cloud.

Multi-sensor

Four environmental/user inputs are interpreted together.

Human-assisted

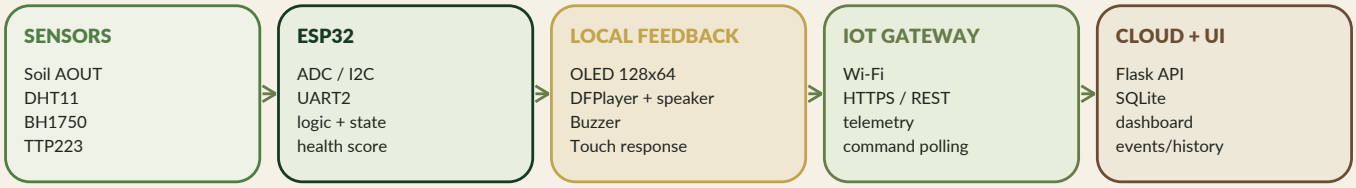
Watering remains a caretaker action; no pump is used.

Bidirectional

Telemetry travels up; queued commands travel back.

3 Proposed System | Functional Blocks

Figure 3.2 | Functional block diagram



Data direction: sensors and events move to the cloud; remote commands return to the ESP32 and are executed locally.

3.3 Block Diagram

- 1. Sensors measure root-zone moisture and surrounding environment.
- 2. ESP32 acquires, filters and validates values.
- 3. Project-defined bands assign states such as DRY, GOOD or LOW LIGHT.
- 4. A composite health score summarises the current state for the UI.
- 5. OLED and voice communicate the interpreted condition locally.
- 6. Telemetry is uploaded to the cloud; remote commands are polled and executed.

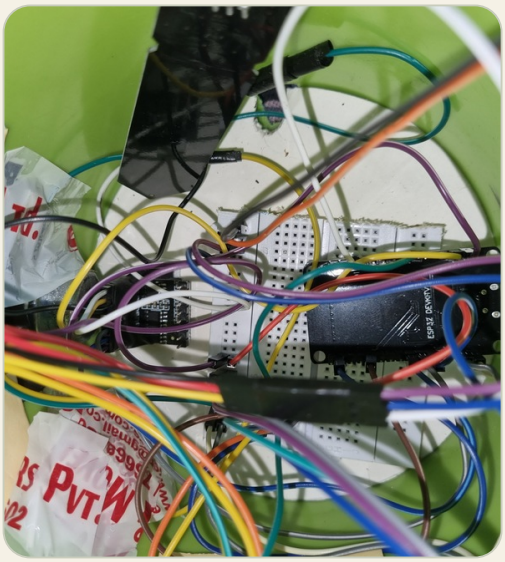


Figure 3.3: Actual internal breadboard assembly photograph supplied by the project author.

3.4 Working Principle

The local firmware runs continuously. Sensor updates are scheduled, not tied to the cloud. Touch actions can wake the OLED, trigger a greeting or start a plant check. Event-driven audio is protected by persistence checks and a cooldown so that a changing sensor value does not repeatedly trigger speech.

3.5 Hardware Architecture

The final physical design places the electronics inside a large outer pot. A smaller inner pot contains the Tulsi and soil; the soil sensor remains in the soil, the DHT11 stays in the air above the soil, and the BH1750 remains exposed at the outer edge for ambient light.

4 Requirements

4.1 Hardware Requirements

Table 4.1 | Hardware used in the prototype

Component	Verified configuration	Role
ESP32 Dev Module / WROOM-32	ESP32-WROOM-32 family; 2.4 GHz Wi-Fi; GPIO21/22/34/4/27/25/16/17	Main controller + IoT
SSD1306 OLED	0.96 in, 128 x 64, I2C, address 0x3C	Local display
BH1750 / GY-302	Digital ambient-light sensing over I2C	Light intensity
Capacitive Soil Moisture V2.0	Analog AOUT -> GPIO34	Root-zone moisture
DHT11 module	Digital DATA -> GPIO4	Temperature + humidity
TTP223 touch module	SIG -> GPIO27	Local user input
DFPlayer Mini + microSD	UART2 + 73 MP3 files; proper 5V supply	Voice output
4 ohm 5 W passive speaker	Connected directly to SPK1/SPK2 in prototype	Audio output
Buzzer	GPIO25 -> +, GND -> -	Short feedback / test
Breadboard + jumpers	Solderless prototyping assembly	Interconnection



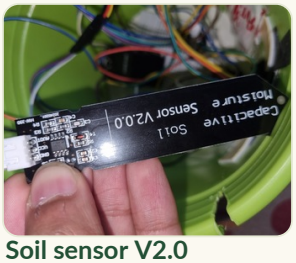
BH1750 / GY-302

User-provided hardware photograph.



DHT11

User-provided hardware photograph.



Soil sensor V2.0

User-provided hardware photograph.



TTP223

User-provided hardware photograph.

Figures 4.1 - 4.4: Actual sensor modules used in PlantPal, supplied by the project author.

4 Requirements | Software, Specifications & Protocols

4.2 Software Requirements

Table 4.2 | Software stack

Software / tool	Use in project
Arduino IDE + ESP32 board support	Firmware development and upload
Adafruit GFX + SSD1306	OLED graphics and I2C display
BH1750 + DHT libraries	Sensor drivers
DFRobotDFPlayerMini	UART audio control
WiFi / WebServer / HTTPS / time	Local AP, cloud link, NTP support
Python + Flask	Backend API and dashboard serving
SQLite	Readings, commands, events and device state
HTML / CSS / JavaScript	Browser-based monitoring and control UI

4.3 Component Specifications

Table 4.3 | Selected component specifications

Part	Key engineering detail
ESP32	2.4 GHz Wi-Fi; 3 UART; 2 I2C; GPIO34 is input-only
BH1750	Digital I2C ambient-light sensor; wide illuminance range
DHT11	0 - 50 C; 20 - 80 %RH typical operating range; low-cost sensor
SSD1306	128 x 64 monochrome OLED; high-contrast local output
DFPlayer Mini	UART-controlled microSD audio; direct SPK1/SPK2 output

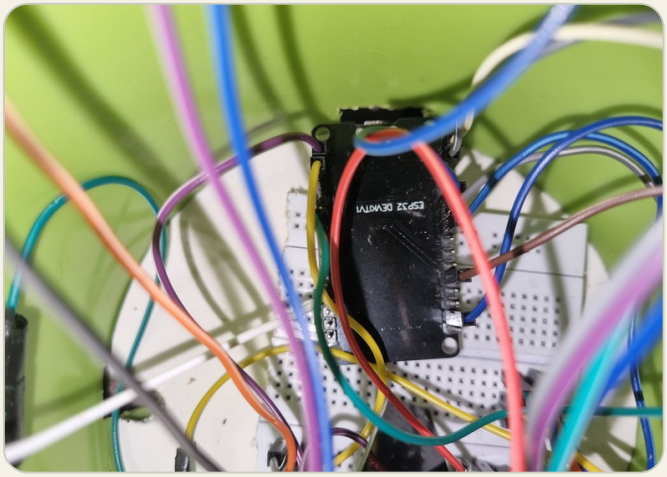


Figure 4.5: ESP32 and breadboard assembly - actual project photograph.

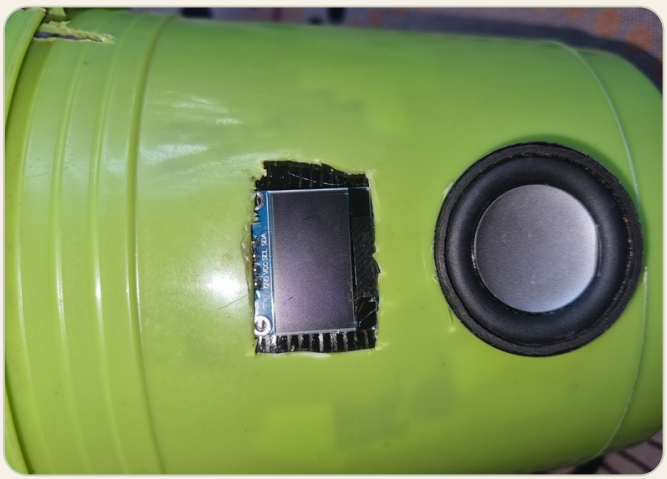


Figure 4.6: OLED and speaker mounted on the outer pot.

Table 4.4 | Communication paths

Layer	Protocol / interface	Use
Local sensor bus	I2C	OLED + BH1750 on GPIO21/22
Audio control	UART2	DFPlayer via GPIO16/17
Sensor input	ADC + digital GPIO	Soil GPIO34; DHT11 GPIO4; touch GPIO27
Network	Wi-Fi	ESP32 station + local PlantPal AP
IoT application	HTTPS / REST + JSON	Telemetry, commands, events
Cloud storage	SQLite	Readings, events, commands and device state

5 Methodology & Implementation

5.1 Methodology

Development followed incremental integration. Each sensor/interface was tested independently before the full local loop was assembled. Audio playback was verified separately, then the cloud layer and dashboard were connected. The final enclosure was built after the wiring and core device behaviour had been exercised.

01	02	03	04	05	06
Requirements	**Hardware**	**Firmware**	**Cloud**	**Integration**	**Validation**
Plant + sensors + local feedback + IoT	ESP32, modules, speaker, enclosure	Sensing, plant model, OLED, touch, audio	Flask + SQLite + REST API	Telemetry + remote commands	Sensor checks, audio, dashboard, enclosure

5.2 Circuit Design

The physical circuit is organised around three signal groups. The OLED and BH1750 share the I2C bus. The soil sensor occupies an ADC input, the DHT11 and touch sensor use digital GPIO, and the buzzer and DFPlayer use dedicated outputs/UART. The audio power path is kept separate from the 3.3 V sensor rail.

5.3 Sensor / Device Integration

The soil sensor is inserted vertically into the Tulsi soil while its connector stays above the soil. The DHT11 remains exposed to air above the soil. The BH1750 is mounted at the outer edge of the pot with its sensing area unobstructed. The final physical enclosure uses thread as a decorative outer wrap around the large pot.

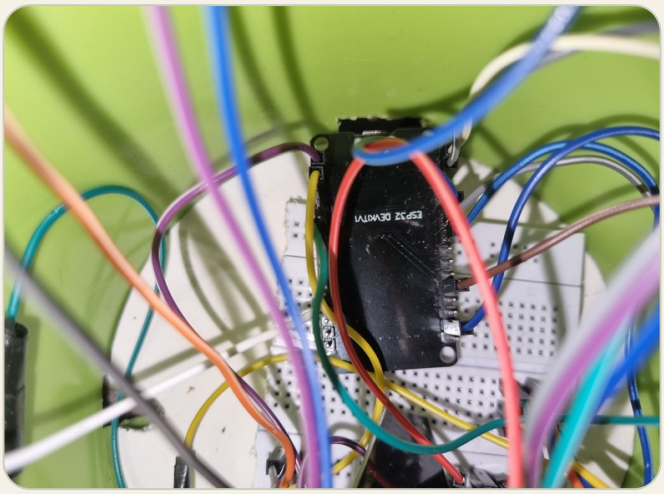


Figure 5.3: Actual ESP32/breadboard assembly supplied by the project author.

Implementation principle

Keep the edge device useful first. The cloud is an extension for telemetry, history and remote control - not the dependency that makes sensing work.

5.4 Data Acquisition

The final firmware uses a 2.5 second sensor cycle, stores validated readings, and updates the plant state model. Soil calibration starts from 3000 raw (dry) and 1200 raw (wet) but is explicitly intended to be calibrated again with the final Tulsi pot.

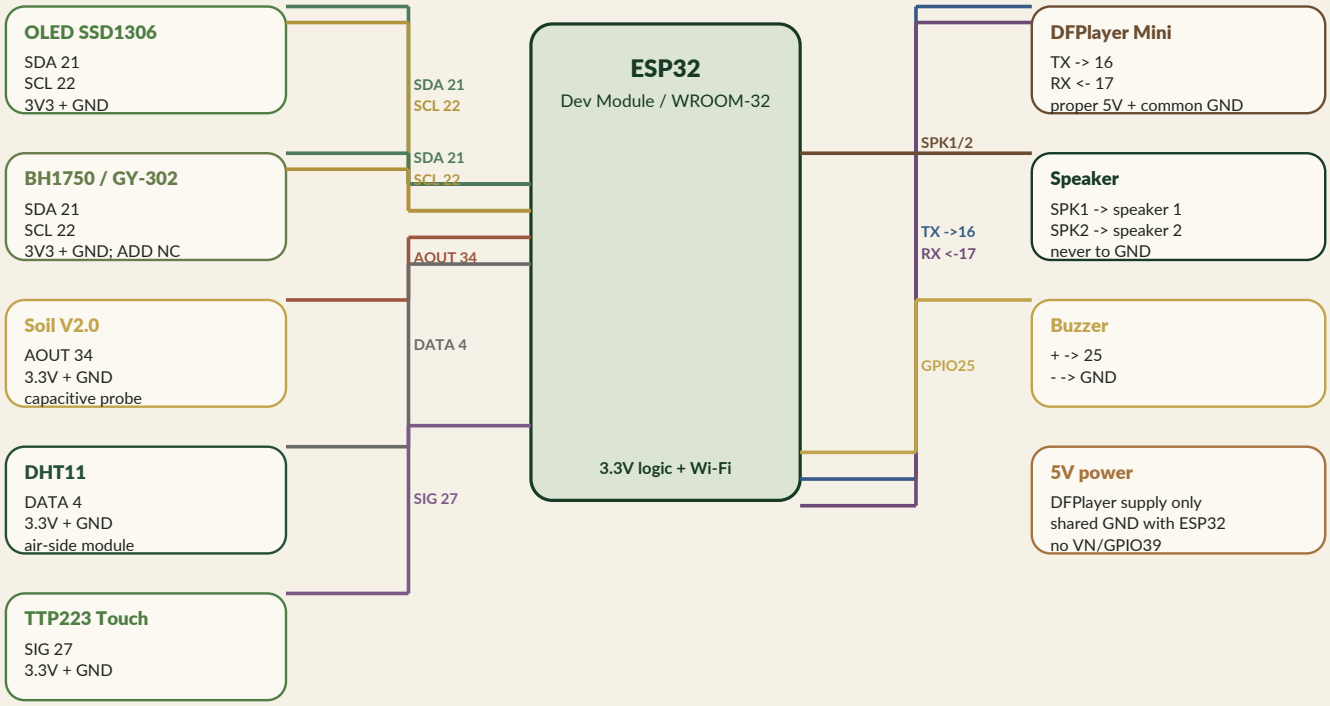
5.5 Data Processing

Condition bands are project-specific engineering thresholds. They classify soil, light, temperature and humidity into readable states and contribute to a composite health score. The score is an interpretation aid, not a laboratory plant-health measurement.

5 Methodology & Implementation

Circuit design | algorithm | IoT communication

Figure 5.1 | Final wiring - Tinkercad-style logical view



Power note: sensors/OLED/touch on 3.3V logic; DFPlayer uses proper 5V; all grounds are common.
GPIO21/22 are shared I2C. GPIO34 is ADC input-only. VN/GPIO39 is not used as a 5V output.

5.6 IoT Communication

Every 5 seconds the ESP32 uploads telemetry to Flask. About every 1.8 seconds it checks pending commands. The backend records readings, events and device state in SQLite.

5.7 Cloud / Edge Implementation

The cloud layer adds remote visibility and control. Local sensing, OLED, touch and voice continue when cloud access is unavailable.

5.8 Algorithm

Read -> validate -> classify -> update interfaces -> sync -> poll -> repeat.

5.9 Flowchart

Figure 5.2 mirrors the executable loop and the local-to-IoT sequence.

Figure 5.2 | Firmware + IoT execution flow



Timing: sensor update 2.5 s | telemetry 5 s | command polling about 1.8 s | condition-audio cooldown 5 min

6 Experimental Setup & Results

6.1 Experimental Setup

Testing used the ESP32 with the final sensor set, DFPlayer and speaker, Tulsi soil/plant, local OLED/touch interfaces, and the two-layer pot enclosure. The local access point supports direct demonstration while the station connection provides the cloud path.

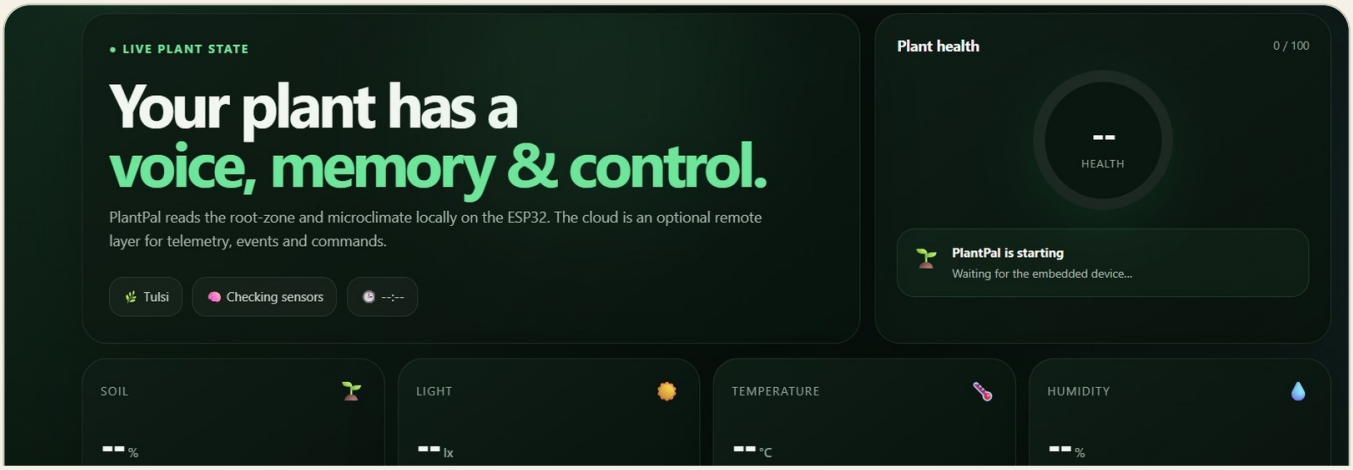


Figure 6.1: PlantPal web dashboard interface — live state, health, sensor cards and plant profile. Screenshot supplied by the project author.

Table 6.1 | Recorded validation checks

Test / interface	Status in project record	Evidence
OLED 128 x 64	Passed	Operated at project I2C address 0x3C.
Soil sensor	Passed	Analog readings obtained on GPIO34.
DHT11	Operational	Final firmware uses GPIO4; transient reads were handled.
TTP223 touch	Passed	Single/double/long gesture handling exercised.
DFPlayer + speaker	Passed	Audio playback and 73-track library exercised.
Cloud telemetry	Passed	Firmware log reported HTTP 201 telemetry upload.
Latest API	Passed	Latest-reading endpoint returned a successful response.

DEVELOPMENT SNAPSHOT | 28 SEP 2026

3291 / 0%	28.8 C	76.2 %	0.0 lx	57 / 100
SOIL RAW / %	TEMPERATURE	HUMIDITY	LUX	HEALTH SCORE

This snapshot is a development record from the cloud path, not a final calibrated plant-health claim. The 0.0 lx value is specifically flagged for BH1750 validation before final submission; soil 0% demonstrates why final dry/wet calibration is required.

7 Applications, Advantages & Limitations

7.1 Applications

- Smart indoor planter
- Educational IoT demonstration
- Environmental monitoring prototype
- Human-assisted plant-care companion
- Base for multi-plant expansion

7.2 Advantages

- Multi-sensor context
- Local operation without cloud
- Voice + OLED + touch feedback
- Two-way IoT communication
- Low-cost prototype architecture

7.3 Limitations

- Soil % depends on calibration and soil composition
- DHT11 has modest accuracy
- BH1750 needs unobstructed light exposure
- 4 ohm 5 W speaker is above the direct-drive recommendation for DFPlayer
- Cloud API is prototype-grade and not production authenticated

7.4 Security Considerations

Actual Wi-Fi credentials are kept in a local ignored configuration file rather than committed to the public repository. The academic API, however, is not presented as a hardened production service. A production deployment should add authenticated users/devices, access control, durable storage, rate limiting, and deployment-appropriate certificate validation.

DESIGN BOUNDARY

PlantPal is a college prototype for embedded and IoT integration. Its measured outputs are intended for demonstration and interpretation - not for medical, agricultural or industrial control decisions.

8. Future Scope

- Dedicated PCB and regulated multi-rail power
- Higher-accuracy climate and soil sensing
- Authenticated dashboard accounts and OTA firmware
- Long-term analytics and downloadable history
- Additional plant profiles and multi-device support

9. Conclusion

PlantPal demonstrates a complete embedded + IoT path: sensors feed an ESP32, the ESP32 interprets the plant state and drives local interfaces, and the same device synchronises that state with a cloud dashboard. The project remains human-assisted by design, while its main limitations are calibration dependence, low-cost sensor accuracy, speaker power matching and prototype-grade security.

References & Appendix

REFERENCES

[1] Espressif Systems. ESP32-DevKitC documentation and board specifications. https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32/esp-dev-kits-en-master-esp32.pdf

[2] Espressif Systems. GPIO & RTC GPIO - ESP32, ESP-IDF Programming Guide. https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/gpio.html

[3] ROHM Semiconductor. BH1750FVI Ambient Light Sensor IC, Rev. D. https://www.datasheetbank.com/en/datasheet-html/190684/ROHM/1page/BH1750FVI.html

[4] Aosong / DFRobot. DHT11 Digital Temperature and Humidity Sensor datasheet. https://image.dfrobot.com/image/data/DFR0067/DFR0067_DS_10.pdf

[5] Adafruit Learning System. Monochrome OLED Breakouts / SSD1306 128x64 wiring guide. https://learn.adafruit.com/monochrome-oled-breakouts

[6] DFRobot. DFPlayer Mini MP3 Player documentation and manual. https://wiki.dfrobot.com/dfplayer_mini_sku_dfr0299

[7] Pallets Projects. Flask Quickstart / Flask documentation. https://flask.palletsprojects.com/en/stable/quickstart/

[8] M. H. Absar et al. “Novel IoT-Based Plant Monitoring System.” Engineering Proceedings, 32(1), 12, 2023. DOI: 10.3390/engproc2023032012. https://www.mdpi.com/2673-4591/32/1/12

[9] “Design of An IoT based Smart Plant Monitoring System.” ICETET-SIP-22, IEEE, 2022. DOI: 10.1109/ICETET-SIP-2254415.2022.9791690. https://ieeexplore.ieee.org/document/9791690/

[10] “Development of an IoT-Based SCADA System for Monitoring of Plant Leaf Temperature and Air and Soil Parameters.” Applied Sciences, 2023, 13(20), 11294. https://www.mdpi.com/2076-3417/13/20/11294

[11] “IoT Based Smart Indoor Plant Health Monitoring System Using FreeRTOS on ESP32 Using Wi-Fi Communication.” ICISC 2025. DOI: 10.1109/ICISC65841.2025.11188353. https://ieeexplore.ieee.org/document/11188353/

[12] “Internet of Things for Crop Farming: A Review of Technologies and Applications.” Commodities, 2023, 2(4), 367-381. DOI: 10.3390/commodities2040021. https://www.mdpi.com/2813-2432/2/4/21

[13] Royal Botanic Gardens, Kew. Plants of the World Online: Ocimum tenuiflorum L. https://powo.science.kew.org/taxon/urn:lsid:ipni.org:names:453130-1/general-information

APPENDIX

```
static const uint8_t PIN_SDA = 21;
static const uint8_t PIN_SCL = 22;
static const uint8_t PIN_SOIL = 34;
static const uint8_t PIN_DHT = 4;
static const uint8_t PIN_TOUCH = 27;
static const uint8_t PIN_BUZZER = 25;
static const uint8_t PIN_DF_RX = 16;
static const uint8_t PIN_DF_TX = 17;

updateSensors();
updatePlantModel();
maybeAutomaticAudio();
serviceCloud();
renderOLED();
```

The complete final firmware is supplied separately as PlantPal_Tulsi_Cloud_Final_v2.ino.

B. Circuit Diagram

See Figure 5.1 on page 12 for the full logical circuit. The actual breadboard assembly is documented with user-provided photographs.

C. Hardware Images



Figure A.1 - A.6: User-provided photographs of PlantPal hardware and assembly.